

2026 開源 AgenticRAG 檔案解析工具完整指南

建構繁體中文 AgenticRAG 系統的最佳選擇已浮現：以 **Docling**（或 **MinerU**）為統一中介表示層、搭配 **HybridChunker** 與 **Chinese-aware** 正規化的堆疊，是目前開源世界最強組合。2025 年的關鍵轉變是解析架構從「規則式 → ML 管線 → 端到端 VLM」快速演進；IBM 的 Docling 在 2026 年初已達 **56K GitHub stars**、Microsoft MarkItDown 衝上 **~92K stars**，而 Chandra/olmOCR 2/PaddleOCR-VL-1.5 則在基準測試上輪流刷新 SOTA。對繁體中文使用者而言，**MinerU 3.1 + PaddleOCR-VL-1.5 / RapidOCR** 是目前最成熟的解法，並應以 OpenCC 與 jieba/CKIPtagger 處理正規化與分詞。本報告梳理 2025-2026 年所有主流開源工具的架構、授權、基準、與實踐整合方式，並給出可直接採用的堆疊建議。

一、PDF 解析工具完整比較

PDF 是 RAG 系統中最難處理的格式，2025-2026 年形成三種清楚的路線：**傳統文字抽取（pypdf 類）**、**ML 管線（Docling/Marker/MinerU）**、以及**端到端 VLM（Chandra/olmOCR/PaddleOCR-VL）**。以下按重要性排序。

Docling（IBM Research，首推的通用解）

github.com/docling-project/docling，**Apache / MIT 授權**，[ChatForest](#) 最新版 2.90.0（2026 年初），**~56K stars**，現已併入 LF AI & Data Foundation。Docling 的最大優勢是提供 `DoclingDocument` 這個**統一中介表示（Unified IR）**——可無損序列化為 Markdown/HTML/JSON/DocTags，並完整保留閱讀順序、標題層級、表格結構（TableFormer 模型）、圖片、公式、程式碼。[Themenonlab](#) [GitHub](#) 輸入格式涵蓋 PDF、DOCX、XLSX、PPTX、HTML、Markdown、[DEV Community](#) AsciiDoc、LaTeX、CSV、PNG/JPEG/TIFF，甚至 WAV/MP3/MP4（[Themenonlab](#) 透過 ASR 擴充）。[ChatForest](#) [GitHub](#)

Docling 內建 **HybridChunker**（結構 + token 對齊的混合分塊器），[GitHub](#) 與 **LangChain**（`langchain-docling / DoclingLoader`）、[GitHub](#) **LlamaIndex**（`DoclingReader / DoclingNodeParser`）、**Haystack**、**Milvus** 都有官方整合。**支援多種 OCR 後端**：EasyOCR（預設）、Tesseract、RapidOCR、OcrMac，並提供 `do_picture_description`（圖片描述 VLM 豐富化），可呼叫 **SmolVLM**、**Granite-Vision-3.2-2B**，或透過 API 呼叫 **GPT-4o / Claude / watsonx**。在 **Procycons 2025** 的永續報告基準測試中，Docling 達到 **97.9% 的複雜表格擷取準確率**。[Procycons](#) [Idp-software](#)

缺點：安裝體積較大（~1 GB+），首次執行需下載 Hugging Face 模型；預設 SmolVLM-256M 產生的圖片描述品質偏弱，[GitHub](#) 生產環境建議換成 **Granite-Vision-3.2-2B** 或遠端 API。

MinerU（上海 AI Lab，繁體中文首選）

github.com/opendatalab/MinerU，[Opendatalab](#) 由中國團隊開發，在**繁體中文 PDF 表現最佳**。最新版 **MinerU 2.5-Pro VLM（1.2B）** 在 OmniDocBench v1.5 上取得 **86.2 分**；[GitHub](#) [GitHub](#) 3.1.0 起授權從 AGPLv3 改為基於 Apache-2.0 的 MinerU Open Source License，[GitHub](#) [GitHub](#) 對商用更友善。[GitHub](#)

MinerU 提供多種 OCR 方案，其中 **ch_lite（PP-OCRv5，18k 字元，中英日繁 + 手寫混合）** 與 **ch_server_v4（PP-OCRv4_server_rec_doc，明確加入繁體中文與日文支援）** 是繁體中文的關鍵差異點。支援**直排文字（豎排）**、印章識別、表格中的公式辨識、109 種語言 OCR。[Opendatalab](#) 特別是，MinerU 支援純 CPU 推論（較慢但可用），並在 Apple Silicon 上提供 `vlm-mlx-engine`（2-3× 速度提升）。[GitHub](#)

建議命令：`mineru --lang ch_server`（繁體）或 `ch_lite`（混合）。Zhihu、CSDN、cnblogs 等中文社群一致推薦 MinerU 為目前 RAG 場景下處理中文 PDF 的首選。

Marker（Datalab / VikParuchuri，英文最佳、GPU 必備）

github.com/datalab-to/marker，** [GitHub](#) ~34K stars**，** [GitHub](#) **GPL-3.0 授權****（商用需買授權）。採用 **Surya OCR**（`surya-ocr ^0.17.1`）+ **pdf-text** 的 ML 管線，[GitHub](#) 已升級至 **block-level OCR 處理**，[GitHub](#) 並加入 `--html_tables_in_markdown` [GitHub](#) 與 `--use_llm`（呼叫 **Gemini/Claude/GPT-4 校正輸出**）等旗標。[GitHub](#) Marker v1.7.5 在 olmOCR-Bench 取得 **76.1 分**，[Idp-software](#) 略勝 MinerU 1.3.10 的 75.8。[Allen AI](#) [GitHub](#)

資源需求：建議 **GPU，每 worker 約 5 GB VRAM**；[GitHub](#) CPU/MPS 可跑但速度劇降（曾有 11 頁文件在 MPS 上耗 540 秒的社群報告）。[GitHub](#) 繁體中文處理品質中等——Surya v3 雖支援中文，但主要在 Latin 語系訓練資料上訓練，[Replicate](#) 繁體中文表現仍不及 PaddleOCR-VL。

pymupdf4llm（最快的純文字抽取方案）

github.com/pymupdf/PyMuPDF，最新 1.27.2.2，**AGPL-3 授權 + 商業授權（Artifex）**。基於 MuPDF C 引擎，[GitHub](#) 提供 **Markdown / JSON / page_chunks=True** 分塊輸出，[GitHub](#) +2 並可整合 **PyMuPDF-Layout**（2025 年 11 月新加的 GNN 版面模型，CPU-only）。**速度極快（毫秒級 / 頁）**，無 **GPU 需求**，自稱比 **VLM 方案便宜 250×**。[GitHub](#) **Pymupdf** 內建 **OCR 偵測邏輯**——自動判斷哪些頁面需要 OCR [PyMuPDF](#) 再委派給 Tesseract 或 **RapidOCR**，[GitHub](#) 相較於全文件 OCR 可節省約 50% 時間。[GitHub](#) [PyMuPDF](#)

適用場景：字型內嵌正確的數位原生 PDF。對於繁體中文 PDF 若出現 `<?>` 佔位符（常見於使用 DFKai-SB、MingLiU 子集化字型且缺 ToUnicode 表的台灣/港 PDF），必須強制 OCR。**AGPL 授權是封閉 SaaS 產品的紅燈**。[PyMuPDF](#)

Unstructured.io（企業 ETL 場景）

github.com/Unstructured-IO/unstructured，**~14.5K stars**，**Apache-2.0**。支援 25+ 檔案格式 [GitHub](#) 並提供 25+ 資料源連接器（[Unstructured](#) **S3**、

SharePoint、Confluence、Salesforce 等）。◦ [Medium](#) 輸出 **Element 列表**（Title、NarrativeText、ListItem、Table、Image...）◦ [scikit-learn +3](#) 並攜帶豐富 **metadata**（filetype、page_number、element_id、languages）。四種策略可選：fast、hi_res（detectron2 版面模型）、ocr_only、auto。◦ [Unstructured](#) [GitHub](#) 在一份 2025 基準測試中簡單表格擷取率接近 100%、複雜表格約 75%。

弱點：預設輸出不是 Markdown；需要系統層依賴（poppler、tesseract、libmagic、libreoffice）；◦ [PyPI](#) 中文品質落後 MinerU/PaddleOCR；部分高階功能鎖在付費 API/Platform。◦ [Unstructured](#)

傳統純 Python 方案（CPU 無依賴）

工具	版本	授權	Stars	RAG 用途
pypdf	6.10.x / 5.7.0	BSD-3	~9K	基本文字 dump、輕量環境
pdfplumber	0.11.7 (2025/6)	MIT	~7K	表格為主的數位 PDF （lattice/stream 兩種演算法）
pdfminer.six	20260107	MIT	~6K	低階版面控制、 CJK 直書

三者皆不支援 OCR、對複雜多欄或學術論文失效，但授權寬鬆、純 Python、零 GPU，常作為 pymupdf4llm 之外的 AGPL 迴避路徑。

MegaParse 狀態警告

[github.com/QuivrHQ/MegaParse](#)，** [GitHub](#) [GitHub](#) ~7.3K stars**，◦ [GitHub](#) Apache-2.0，但自 2025 年 2 月 v0.0.55 後未再發布，◦ [GitHub](#) 許多 2025 年 bug 仍未解，repo 多次重組。不建議用於新的生產專案。

OCR 引擎子系統

工具	最新版	授權	繁體中文	CPU/GPU
PaddleOCR-VL-1.5 (Jan 2026)	1.5	Apache-2	✅ 109 語言	GPU + vLLM
PP-StructureV3	PaddleOCR 3.0.3	Apache-2	✅ 最佳	皆可
PP-OCrv5	2025/5	Apache-2	✅ 單模型含繁體	皆可
RapidOCR	3.8.1 (2026/4)	Apache-2	✅ 優秀	CPU (ONNX)
Surya v3	2025	GPL-3 (weights 商用 OK)	改善，但不及 Paddle	GPU 建議
Tesseract	5.5.2 (2025/12)	Apache-2	可用（chi_tra/chi_tra_vert）	CPU

關鍵事實：PP-OCrv5 是單一模型同時支援繁體、簡體、英日、手寫，比 v4 準確率提升 13%；◦ [PyPI +2](#) PaddleOCR-VL-1.5 宣稱在 OmniDocBench v1.5 達 **94.5%**；◦ [AMD +4](#) Tesseract 在 2026 年已退居「備援」地位，品質明顯落後任何 ML-based OCR。

二、非 PDF 格式的解析方案

2025-2026 年的統一轉換工具三強鼎立：**MarkItDown**（~92K stars，MIT）、**Docling**（~56K stars，MIT）、**Unstructured**（~14.5K stars，**Apache-2.0**），三者皆在底層包裝 python-docx、python-pptx、openpyxl、mammoth 等成熟的格式專用函式庫。

MarkItDown（Microsoft，LLM-first Markdown）

[github.com/microsoft/markitdown](#)，由 Microsoft AutoGen Team 維護，◦ [ChatForest](#) 最新 0.1.4（2025/12）/ 0.1.5b1（2026/2）。輸入涵蓋 PDF（text-only）、DOCX、PPTX、XLSX、HTML、CSV、JSON、TXT、EPub、圖片（+ EXIF + 可選 LLM OCR）、音訊（可選轉錄）、ZIP、YouTube URL、◦ [AdwaitX](#) [Brightcoding](#) Outlook .msg。輸出單一統一 **Markdown 字串**，保留標題、清單、表格（GFM）、連結、粗體/斜體、程式碼。◦ [GitHub](#)

內部後端：DOCX 用 **mammoth**、PPTX 用 **python-pptx**、XLSX 用 pandas/openpyxl、PDF 用 pdfminer.six、HTML 用 BeautifulSoup/markdownify。接受 llm_client 參數（OpenAI/Azure OpenAI）讓 GPT-4o 產生圖片描述；◦ [GitHub](#) 另有 markitdown-mcp（2025/4）暴露給 Claude Desktop 等 MCP 客戶端。** [AdwaitX](#) Chunk 友善度最高**（# H1 只佔 3 tokens，HTML 等效則要 23 tokens）。

主要弱點是 PDF：無內建 OCR（需 plugin 或 Azure Document Intelligence），◦ [GitHub](#) 對複雜 PDF 常剝除標題/清單結構，某第三方評測指其複雜 PDF 成功率僅 25%（◦ [AdwaitX](#) 軼聞數據，僅供參考）。

DOCX 專用

- **python-docx** 1.2.0（2025/6，MIT，◦ [PyPI](#) ~5.5K stars）：低階 OOXML 物件模型，是 MarkItDown/Docling/Unstructured 的共同底層。不直接輸出 Markdown。
- **mammoth** 1.11.0（2025/9，BSD-2）：DOCX → 語義 **HTML**（將 Word 樣式 Heading 1 映射為 <h1>），官方已棄用 Markdown 直接輸出，建議轉 HTML 再經 markdownify。◦ [GitHub](#) MarkItDown 內部的 DOCX 後端。

- **docx2txt 0.9**：[Libraries.io](#) **LangChain** `Docx2txtLoader` 底層，僅純文字，[Libraries.io](#) 無結構保留，目前維護冷清。[Snyk](#)

RTF 與其他

- **striprtf 0.0.29** (BSD-3)：純文字輸出，支援 ANSI codepage (繁體 cp950)。無結構保留；適合醫療/法律舊檔。
- **py pandoc 1.15** (2025/1，MIT；背後 Pandoc GPL-2)：通用萬能轉換器 (40+ 輸入 × 60+ 輸出)。RTF/ODT/LaTeX/reStructuredText/Org/FictionBook 等冷門格式的最後救星；`py pandoc_binary` 自帶二進位。

Markdown 解析 (RAG 入料後常見需求)

- **markdown-it-py 4.0.0** (2025/8，MIT)：100% CommonMark + GFM 擴充，[PyPI](#) token stream + `SyntaxTreeNode` 可精準在 H2/H3 邊界切分。[PyPI](#) `markdown-it-pyrs` Rust 綁定快 20x。[PyPI](#)
- **mistune 3.2.0** (2025/12，BSD-3)：**Python 最快** (mistune 15s vs markdown-it-py 42s vs python-markdown 48s，spec.md 30 iterations)，但非 CommonMark 完全相容。
- **Python-Markdown 3.10.2** (2026/2，BSD-3)：MkDocs/Sphinx 生態系相容首選。

Office 檔案

- **openpyxl 3.1.5** (MIT)：XLSX 讀寫；[GitHub](#) `MarkItDown/Docling` 的 XLSX 後端。建議搭配 `defusedxml` 防 XXE 攻擊。[GitHub](#)
- **python-pptx 1.0.2** (MIT，[PyPI](#) ~2.4K stars)：PPTX 讀寫；完整 shape tree、表格、圖表、備註。
- **xlrd 2.0.1**：僅支援舊版 `.xls` (2020 年後移除 XLSX 支援)。

三大統一方案對照

維度	MarkItDown	Docling	Unstructured
Stars (2026)	~92K	~56K	~14.5K
授權	MIT	MIT	Apache-2.0
PDF 品質	⚠️ 弱	✅✅ 強	✅
表格結構	GFM 管道表	TableFormer AI	Table Transformer
內建 Chunker	❌	HybridChunker	<code>by_title</code>
圖片 OCR	plugin/LLM	內建 EasyOCR	Tesseract
LangChain 官方 loader	社群	✅ <code>langchain-docling</code>	✅
安裝體積	小 (~50-200 MB)	大 (~1 GB+)	中 (+ 系統依賴)
最佳用途	快速通用預處理	高保真 RAG	企業 ETL 多連接器

三、RAG 評估維度關鍵比較

針對 RAG 場景，結構保留能力與分塊友善度是兩大決勝因子。下表按 10 個維度評比主要候選：

工具	結構保留	Markdown 輸出	表格品質	圖片/OCR	速度	GPU	繁中	Chunker	LangChain	授權
Docling	★★★★★	✅ + DocTags + JSON	★★★★★	EasyOCR/VLM	中	選	✅	HybridChunker	官方	MIT
MinerU 2.5	★★★★★	✅ + JSON	★★★★	PP-OCRv5/VLM	中	選	★★★★★	間接	社群	MinerU OSL
Marker	★★★★	✅	★★★★	Surya	慢	需	★★★	間接	社群	GPL-3
MarkItDown	★★★	✅	★★★	LLM plugin	快	❌	✅ (Unicode)	❌	社群	MIT
Unstructured	★★★★	間接	★★★★	Tesseract/ML	中	選	中	<code>by_title</code>	官方	Apache-2
					極					

pymupdf4llm	★★★	✓	★★★	Tesseract	快	✗	✓ (數位)	page_chunks	官方	AGPL-3
pypdf/pdfplumber	★★	✗	中/★★★★	✗	快	✗	部分	✗	LangChain loader	BSD/MIT

三個實務要點：其一，**DoclingDocument** 是目前最豐富的開源文件 IR，可同時序列化為 Markdown（給 LLM）、JSON（給向量庫 metadata）、以及具 bbox 的 DocTags（給 visual grounding 引用介面）。其二，授權風險常被忽視：Marker GPL-3 與 pymupdf4llm AGPL-3 在封閉 SaaS 產品中可能強制開源——Datalab 與 Artifex 都有商業授權可買，務必事前確認。其三，繁體中文的唯一明確贏家是 **MinerU 搭 PaddleOCR 系**，其他工具靠底層 OCR 繞過，品質差距明顯。

四、AgenticRAG 實踐架構

Router 設計：以 MIME 為主、以 canonical IR 為核心

核心原則是：統一於一個 **canonical representation**（推薦 **DoclingDocument**），而非每種格式各自走自己的路徑，否則程式碼路徑會組合爆炸。使用 `python-magic` 偵測 MIME，然後分派：

```
import magic, mimetypes
from pathlib import Path
from docling.document_converter import DocumentConverter, PdfFormatOption
from docling.datamodel.base_models import InputFormat
from docling.datamodel.pipeline_options import (
    PdfPipelineOptions, granite_picture_description)

PDF_OPTS = PdfPipelineOptions(
    do_ocr=True, do_table_structure=True,
    do_picture_classification=True, do_picture_description=True,
    picture_description_options=granite_picture_description,
    generate_picture_images=True, images_scale=2.0,
)
converter = DocumentConverter(
    format_options={InputFormat.PDF: PdfFormatOption(pipeline_options=PDF_OPTS)}
)

def parse_any(path: Path):
    mime = magic.from_file(str(path), mime=True) or mimetypes.guess_type(path)[0]
    if mime in {"text/csv", "application/json"}:
        return load_tabular(path) # 自訂路徑
    # Docling 原生處理 PDF / DOCX / PPTX / XLSX / HTML / MD / images / audio
    return converter.convert(str(path)).document # → DoclingDocument
```

`Unstructured.partition()` [GitHub](#) 與 `LlamaIndex SimpleDirectoryReader` 都提供類似的自動分派機制，但都不如自建 `dispatcher` 能精確控制各格式的 `pipeline` 參數。注意：Unstructured 在檔案缺 magic bytes 時（例如 Confluence 匯出的 `.doc`）會 fallback 到副檔名，[GitHub](#) 若以 bytes stream 處理務必顯式傳入 `content_type`。

分塊策略：HybridChunker 是 2025-2026 新標準

2025 年的分塊共識與 Vecta、NVIDIA、Weaviate 的基準測試一致：

- **RecursiveCharacterTextSplitter** (`chunk_size` 400-512 tokens、10-20% overlap) 仍是最穩健的通用預設；Vecta 2026/2 基準在 50 篇學術論文上取得 69% 準確率，[Firecrawl](#) 7 種策略中排名第一。
- **語義分塊 (SemanticChunker)** 在敘事散文上能 +9% 召回率，[Firecrawl](#) 但在密集結構化文件上可能掉 15 個百分點——只對自由流動的文字有效。
- **結構化分塊** (Docling `HierarchicalChunker`、Unstructured `by_title`、LangChain `MarkdownHeaderTextSplitter`) 是企業結構化文件首選。
- **HybridChunker (Docling)**：先做階層式切分，再以 `tokenizer-aware` 拆分/合併到目標 token 預算——** [GitHub](#) IBM、OpenSearch、Red Hat RH [Red Hat EL AI 1.3](#) 皆評為目前 PDF/DOCX 最佳**。
- **Contextual Retrieval (Anthropic, 2024/9)**：用 LLM 為每個 chunk 產生 50-100 token 的文件脈絡前綴，報告單獨減少 **35% 召回失敗、配合 BM25 達 49%、再加 reranker 達 67%**；[DataCamp](#) 以 prompt caching 成本約每百萬文件 tokens \$1.02 美元。
- **Late Chunking (Jina AI, ICLR 2025 workshop)**：先把整份文件做 token 級 embedding 再按 chunk 邊界 pool，解代詞/指涉問題，+10-12% 召回；需要長上下文 embedding 模型 (jina-v3、bge-m3)。

務必讓 chunker 對齊 embedding 模型的 tokenizer：

```
from docling_core.transforms.chunker.tokenizer.huggingface import HuggingFaceTokenizer
from transformers import AutoTokenizer
from docling.chunking import HybridChunker
```

```
tok = HuggingFaceTokenizer(  
    tokenizer=AutoTokenizer.from_pretrained("BAAI/bge-m3"),  
    max_tokens=512,  
)  
chunker = HybridChunker(tokenizer=tok, merge_peers=True)
```

Metadata 保留：為 Agent 可查詢性而設計

每個 chunk 都應攜帶 doc_id、source_uri、binary_hash、page_numbers[]、section_path[]（H1→H2→H3 路徑）、element_type（paragraph/table/figure/code）、bbox、parent_id、doc_items_refs（Docling 指回原樹節點的指標）、effective_date、owner。向量庫建議選具備強大 payload 篩選的 Qdrant / Weaviate / Milvus / OpenSearch，讓 Agent 可下 filter: element_type == "table" AND section_path CONTAINS "Revenue"。

多模態 PDF 的決策樹

- 1. 文字可抽取、版面簡單 → 原生抽取（pymupdf4llm）。
- 2. 文字可抽取、版面複雜（多欄、表格） → Docling hi_res / Unstructured hi_res。
- 3. 掃描影像型 PDF → OCR（Docling + RapidOCR / PaddleOCR / Tesseract）。
- 4. 圖表、示意圖的「意義在圖像裡」 → VLM 圖說（Docling do_picture_description + Granite-Vision-3.2-2B 或 GPT-4o API）。
- 5. 整頁視覺至上（簡報、金融文件、表單） → 跳過文字抽取，改用 ColPali / ColQwen2.5 頁面影像嵌入。

實務中最佳方案是混合：Docling 處理文字、VLM 為圖片加說明、ColPali 為視覺重的頁面建立平行索引——Agent 可依查詢選擇檢索通道。注意 Docling 預設的 SmolVLM-256M 圖說品質偏弱（常產出「這張圖展示一張示意圖」），生產請改用 Granite-Vision-3.2-2B 或 GPT-4o。

LlamaIndex IngestionPipeline 範例（推薦骨架）

```
from llama_index.core.ingestion import IngestionPipeline, IngestionCache  
from llama_index.core.ingestion.cache import RedisCache  
from llama_index.node_parser.docling import DoclingNodeParser  
from llama_index.extractors import TitleExtractor  
from llama_index.embeddings.huggingface import HuggingFaceEmbedding  
from llama_index.vector_stores.qdrant import QdrantVectorStore  
  
pipeline = IngestionPipeline(  
    transformations=[  
        DoclingNodeParser(chunker=HybridChunker(tokenizer=tok)),  
        TitleExtractor(),  
        HuggingFaceEmbedding("BAAI/bge-m3"),  
    ],  
    docstore=SimpleDocumentStore(), # 提供 upsert / dedup  
    cache=IngestionCache(cache=RedisCache.from_host_and_port("localhost", 6379)),  
    vector_store=QdrantVectorStore(client=qc, collection_name="rag"),  
    docstore_strategy="upserts", # or duplicates_only / upserts_and_delete  
)  
nodes = pipeline.run(documents=docs, num_workers=4)
```

AgenticRAG 專屬模式

arXiv 2501.09136 (v4, 2026/4) 與 Anthropic、NVIDIA、RAGFlow 0.21 的共識是：「檢索從固定預處理步驟變成 Agent 可迭代呼叫的工具」。這有兩個連鎖效應：（一）需要更豐富的 metadata；（二）多粒度索引（parent-child chunks、page-level + section-level + sentence-level）平行建立，讓 Agent 可依子任務選擇合適粒度。

推薦實作的 Agent 模式：**Query-planning Agent** 先查文件目錄索引（TOC、章節樹）決定查哪份文件；**Corrective RAG (CRAG)** 會評分檢索結果並在失敗時改寫查詢；**Parent-child Retrieval** 以小 child chunk 做精確召回、回傳 parent section 作為 LLM 上下文（Docling 階層天然適用）；**Tool-based Retrieval** 把 search_section()、get_table(doc_id, page)、get_figure_caption() 暴露為 Agent 獨立工具而非單一 retrieve()。

五、2025-2026 年的新趨勢

VLM 浪潮：從管線到端到端

2025 年是 VLM 解析器全面崛起的一年，關鍵發布時序如下：

發布	組織	日期	參數	亮點
o1mOCR 0.158	Allen AI	2025/2	7B	Qwen2-VL 蒸餾自 GPT-4o

SmolDocling-256M	IBM + HuggingFace	2025/3	256M	引入 DocTags 格式
Dolphin	ByteDance	2025/5	0.3B	「分析-再-解析」ACL 2025
Granite-Docling-258M	IBM	2025/9	258M	SigLIP2 + Granite，FinTabNet TEDS 0.97
Chandra 1	Datalab	2025/10	9B	olmOCR-Bench 83.1
DeepSeek-OCR	DeepSeek	2025/10	3B	「Contexts Optical Compression」
olmOCR 2	Allen AI	2025/10	7B	RLVR + GRPO 訓練
MonkeyOCR v1.5	—	2025/11	—	視覺一致性 RL
PaddleOCR-VL-1.5	Baidu	2026/1	0.9B	OmniDocBench v1.5 94.5%
Chandra 2	Datalab	2026/3	4B	olmOCR-Bench 85.9

基準榜單（2026 年初）

olmOCR-Bench（1,403 PDF / 7,010 單元測試，Allen AI 維護）目前榜首：Infinity-Parser2-Pro **86.7**、Chandra 2 **85.9**、dots.mocr 83.9、LightOnOCR-2-1B 83.2、Chandra 1 83.1、olmOCR-2-7B 82.4、Marker v1.7.5 76.1、MinerU 1.3.10 75.8。

OmniDocBench v1.5（上海 AI Lab）則被 LlamaIndex 明確稱為「飽和」：GLM-OCR（Z.ai）94.6%、PaddleOCR-VL-1.5 >94%、FireRed-OCR 92.94%、Gemini 3 Pro 90.3%、MonkeyOCR v1.5、MinerU 2.5。榜單中多數結果**自報**，第三方驗證少，應保留戒心。

產品技術棧推測（僅憑公開資訊）

- **NotebookLM (Google)** 透過 Gemini + RAG，解析層應走 Vertex AI Search 的 Digital Parser / OCR Parser / Layout Parser 三段式，再以 Gemini 家族模型做文字理解。
- **ChatGPT 檔案上傳** 技術棧最簡單——Advanced Data Analysis 沙箱直接用 PyMuPDF / pdfplumber / python-docx，掃描檔則 **GPT-4o 本身就是 VLM**，不另設解析器。
- **Claude Projects** 同理——Claude 3.5/3.7/Opus 4 原生讀 PDF（文字 + 渲染頁面影像），配合 2024/9 揭露的 Contextual Retrieval。
- **Perplexity** 無官方揭露，推測仍以 pdfium / PyMuPDF 做底層文字抽取 + LLM 清理。

共同模式：前沿大模型公司把 **LLM 本身當解析器**；開源陣營則組裝 Granite-Docling、olmOCR、Chandra 等小型專用 VLM 置於文字 LLM 之前。

結構化輸出新標準：DocTags

SmolDocling 論文（ICCV 2025）導入的 **DocTags** 是 XML 風格標記，將「內容」與「結構」分離並內嵌 bbox、圖表-說明關係、閱讀順序；比 **Markdown/HTML 更忠實、token 更省、RAG grounding 品質更好**。Granite-Docling 完整採用，預期 2026 年下半年進一步擴散。

ColPali / ColQwen2.5 視覺檢索

2024/6 發布、ICLR 2025 論文的 **ColPali** 以 PaliGemma-3B 直接嵌入頁面影像，使用 ColBERT-style late-interaction（MaxSim）匹配，**完全跳過 OCR 與分塊**。**ColQwen2**（vidore/colqwen2-v1.0）換用 Qwen2-VL-2B 後在動態解析度上 +5.3 nDCG@5；**ColQwen2.5-v0.2** 於 2025 年末釋出。Weaviate、Qdrant、Vespa、Elasticsearch 都已支援 multi-vector MaxSim。代價是**每頁約 1000 個向量**，儲存成本 10-100×；適用於 <10K 頁的高價值視覺重語料（金融簡報、工程圖、醫學影像）。

RLVR 訓練配方擴散

olmOCR 2（2025/10）首次對 OCR 使用 **GRPO + 二元單元測試獎勵** 的 RL 訓練，產生合成 HTML 驗證案例。MonkeyOCR v1.5 迅速跟進「視覺一致性 RL」。
DeepSeek-OCR 則提出「Contexts Optical Compression」——把長文字編成緊湊的視覺 token（7-20× 壓縮率），重新定義 OCR 為 Agent 長文件推理的壓縮原語。

六、繁體中文與實務推薦堆疊

繁體中文的獨特陷阱

繁體中文 PDF 有**五個致命陷阱**，大部分開源工具預設會中招：

1. **子集化字型 + 無 ToUnicode 表**（常見於台灣政府、司法、學術 PDF，使用 DFKai-SB / MingLiU 變體）→ pymupdf / pdfminer / pdfplumber 都會輸出 <? > 佔位符或錯誤閱讀順序。**解法**：強制 OCR（force_full_page_ocr=True / --method ocr / --force_ocr）。
2. **直排文字**：僅 **MinerU 2.5** 與 **PP-StructureV3** 原生支援；Tesseract 需 chi_tra_vert；其餘工具幾乎都讀錯。
3. **全形/半形符號混用**：破壞精確比對檢索，需入庫前做 **Unicode NFKC 正規化**。
4. **PaddleOCR v3/v4 預設 lang=ch 是簡體字典**，讀繁體會產生亂碼（台→合、臺→壹）。**v5 起統一單模型可用 lang=ch**；v3/v4 必須明確指定

lang=chinese_cht 。

5. Tesseract 只裝 chi_sim 讀繁體 → 亂碼，必須另裝 chi_tra.traineddata （或社群強化版 gumblex/tessdata_chi）。

必備工具配件

- **OpenCC** (BYVoid/OpenCC，pip install opencc)：簡繁正規化。推薦 config s2twp（考慮台灣慣用語）進行入庫前正規化，避免簡/繁變體在向量空間中分裂。
- **jieba + dict.txt.big**（繁中詞典）或 **CKIPTagger**（中研院）：BM25/hybrid 檢索的分詞器。CKIPTagger 在長詞處理上明顯勝過 jieba 預設繁體字典。
- **bge-m3**（多語言、支援簡繁英、同時提供 dense + sparse + ColBERT 三種表示）或 **bge-large-zh-v1.5**：embedding 首選。
- **bge-reranker-v2-m3**：reranker，配 Anthropic Contextual Retrieval 是目前已知 SOTA 組合。

入門推薦堆疊（繁體中文 AgenticRAG，2026）

解析層： MinerU 3.1 (pipeline 後端，--lang ch_server) for PDF/DOCX/PPTX/XLSX
pymupdf4llm 作為乾淨數位 PDF 的快速路徑
OCR 備援： RapidOCR (PP-OCrv5 ONNX) 作為純 CPU 伺服器的 OCR 後備
正規化： OpenCC (s2twp) + Unicode NFKC + 全形/半形統一
分塊： LangChain RecursiveCharacterTextSplitter，分隔符
["\n\n", "\n", "。", "!", "?", ";", ", ", "\n", " ", ""]
目標 400-600 漢字 (≈ 600-1200 tokens)、10-15% overlap
分詞： jieba + dict.txt.big + Taiwan userdict，或 CKIPTagger
嵌入： BAAI/bge-m3 (dense + sparse 一體)
向量庫： Qdrant 或 Milvus
LLM： Qwen2.5/3、GPT-4o、或 Claude 3.7+（皆原生繁中）

進階堆疊（GPU 可用、追求 SOTA）

解析主力： MinerU 2.5-Pro VLM (GPU) + PaddleOCR-VL-1.5 並行驗證器
版面分析： PP-StructureV3（表格重 / 公文格式）
表格： MinerU StructEqTable（複雜公式）+ DeepDoc（合併儲存格）
直排處理： MinerU 2.5 原生，或 Tesseract chi_tra_vert 預處理
多模態： Docling do_picture_description + Granite-Vision-3.2-2B
對視覺重負加平行 ColQwen2.5 索引
分塊： HybridChunker (bge-m3 tokenizer 對齊)
+ Anthropic Contextual Retrieval (Claude Haiku + prompt caching)
檢索： Hybrid (BM25 with jieba + dense bge-m3) + bge-reranker-v2-m3 RRF fusion
Router： Chandra 處理難掃描、MinerU 處理一般、pymupdf4llm 處理乾淨數位
框架： LlamaIndex IngestionPipeline + LangGraph 編排

部署取舍

場景	建議堆疊	預期資源
純 CPU、低延遲	pymupdf4llm + RapidOCR + MarkItDown	2-4 GB RAM
CPU 為主、品質優先	Docling + RapidOCR + HybridChunker	4-8 GB RAM，模型 ~1 GB
繁中為主、CPU OK	MinerU pipeline + RapidOCR	8 GB RAM
繁中 + GPU 可用	MinerU 2.5 VLM 或 PaddleOCR-VL-1.5	8-12 GB VRAM
SOTA 英文	Chandra 2 或 olmOCR 2	16-24 GB VRAM
視覺重文件	ColQwen2.5 + Qwen2.5-VL	16 GB+ VRAM

結論：選型心法與再三強調的三個重點

回到 2026 年初的開源解析生態，有三個非做不可的決策：第一，請把 **DoclingDocument** 當成你的 **canonical IR**——它是目前唯一同時具備完整結構、表格、圖片描述、visual grounding 能力、並可無損序列化為 JSON 的開源 IR，不要讓每種檔案格式各自通往分塊器。第二，**繁體中文使用者必須繞過「歐美中心」的 Marker / Surya / Unstructured 路徑**，改以 MinerU + PaddleOCR-VL 為核心；這不是偏好問題，而是 OCR 訓練資料分佈的客觀差異。第三，**分塊不再是單次的前處理**——Contextual Retrieval (Anthropic) + Hybrid BM25/dense + reranker 才是 2025-2026 年實測證明的 SOTA 三件組，把它內化到 Ingestion pipeline 而非檢索時臨時加料。

值得留心的新訊號：DocTags 正在取代 Markdown 成為 LLM-first 的結構化輸出；Granite-Docling-258M 證明 <300M 參數的小 VLM 在表格識別上已可追平 7B 模型；ColPali/ColQwen2.5 已讓「完全不 OCR」成為視覺重文件的可行選項；RLVR 訓練配方 (olmOCR 2) 把 OCR 品質提升曲線重啟。開源陣營與前沿實

驗室的差距仍然存在，但從 2025 到 2026 的節奏看，這個差距縮小的速度比過去三年的總和還快——現在建構 AgenticRAG，比 2023 年要合理且負擔得起許多。